IHM Java

Pr. Mohammed GABLI

FSO- Oujda

medgabli@yahoo.fr

SMI-S5
2023-2024

Plan

- Introduction à JAVA Swing
- Évenements
- Boutons et Textes
- Cases à cocher et Radio boutons
- Graphique 2D
- Animation
- Architecture MVC
- TPs : Applications en Netbeans et Eclipse

Introduction (rappel)

- Les applications informatiques nécessitent des échanges d'informations entre l'utilisateur et la machine ;
- Cet échange => interface utilisateur (UI en anglais) en mode texte (ou console) ou en mode graphique ;
- Une interface graphique => plusieurs fenêtres => divers composants graphiques : `boutons, listes déroulantes, menus, champ texte, etc`;
- Les interfaces graphiques sont souvent appelés **GUI** (Graphical User Interface).

Introduction (rappel)

- Les applications informatiques nécessitent des échanges d'informations entre l'utilisateur et la machine ;
- Cet échange => interface utilisateur (UI en anglais) en mode texte (ou console) ou en mode graphique ;
- Une interface graphique => plusieurs fenêtres => divers composants graphiques : `boutons, listes déroulantes, menus, champ texte, etc`;
- Les interfaces graphiques sont souvent appelés **GUI** (Graphical User Interface).

Introduction (rappel)

- Les applications informatiques nécessitent des échanges d'informations entre l'utilisateur et la machine ;
- Cet échange => interface utilisateur (UI en anglais) en mode texte (ou console) ou en mode graphique ;
- Une interface graphique => plusieurs fenêtres => divers composants graphiques : `boutons, listes déroulantes, menus, champ texte, etc`;
- Les interfaces graphiques sont souvent appelés **GUI** (Graphical User Interface).

Introduction (rappel)

- Les applications informatiques nécessitent des échanges d'informations entre l'utilisateur et la machine ;
- Cet échange => interface utilisateur (UI en anglais) en mode texte (ou console) ou en mode graphique ;
- Une interface graphique => plusieurs fenêtres => divers composants graphiques : `boutons, listes déroulantes, menus, champ texte, etc`;
- Les interfaces graphiques sont souvent appelés **GUI** (Graphical User Interface).

Introduction (rappel)

Bibliothèques utilisées :

- `AWT` (Abstract Window Toolkit, JDK 1.1) ;
- `Swing` (JDK/SDK 1.2) ;
- `JavaFX`.

Introduction

- Actuellement `JavaFX` est la bibliothèque graphique remplaçante de `Swing` et de `AWT` ;
- La première version de `JavaFX` 1.0 a été créée en 2008 ;
- Dans les versions courantes 1.8, les trois bibliothèques sont toujours accessibles, mais seule `JavaFX` va, dans le futur, avoir de nouvelles fonctionnalités.

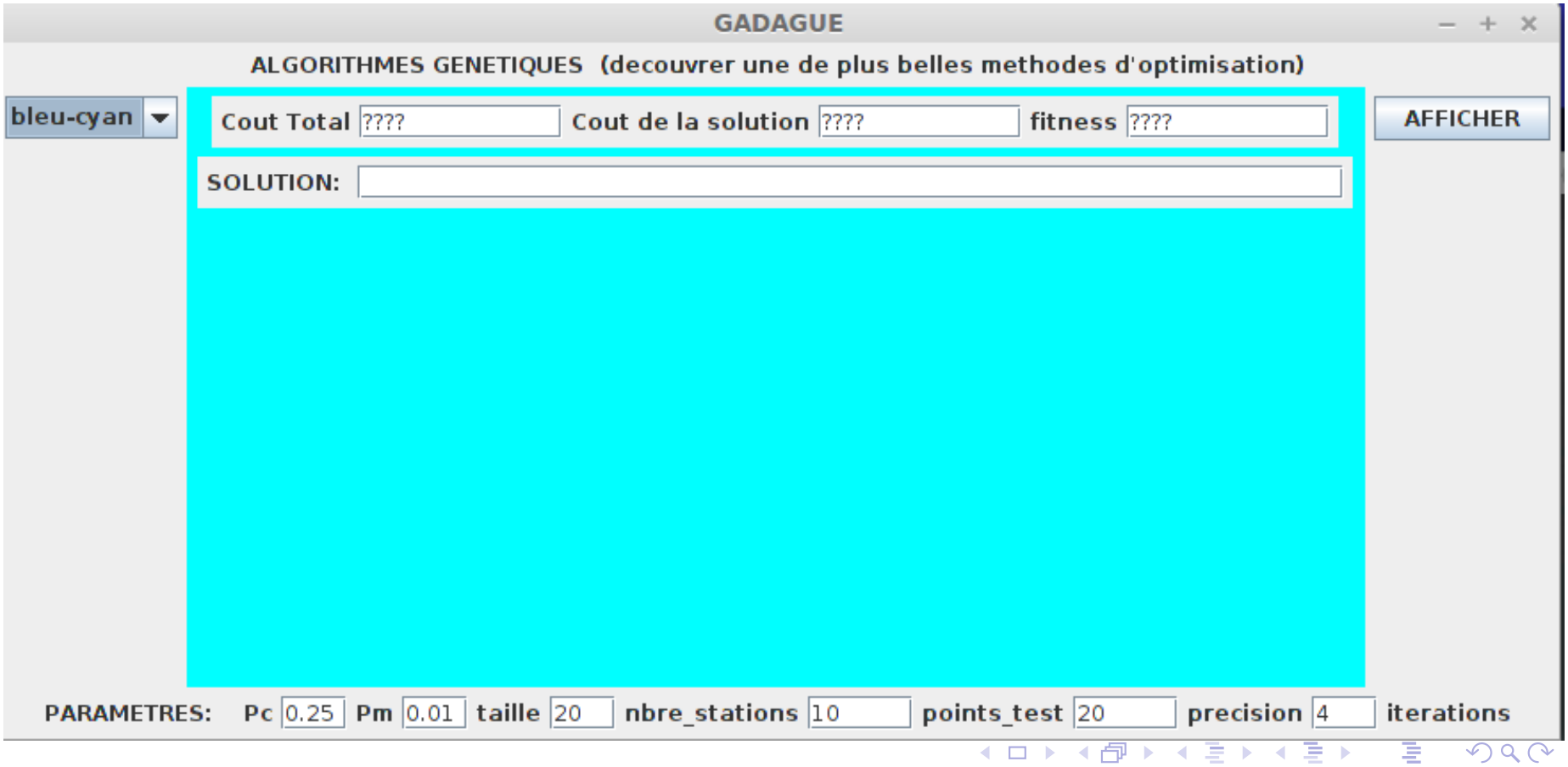
Introduction

- Actuellement `JavaFX` est la bibliothèque graphique remplaçante de `Swing` et de `AWT` ;
- La première version de `JavaFX` 1.0 a été créée en 2008 ;
- Dans les versions courantes 1.8, les trois bibliothèques sont toujours accessibles, mais seule `JavaFX` va, dans le futur, avoir de nouvelles fonctionnalités.

Introduction

- Actuellement `JavaFX` est la bibliothèque graphique remplaçante de `Swing` et de `AWT` ;
- La première version de `JavaFX` 1.0 a été créée en 2008 ;
- Dans les versions courantes 1.8, les trois bibliothèques sont toujours accessibles, mais seule `JavaFX` va, dans le futur, avoir de nouvelles fonctionnalités.

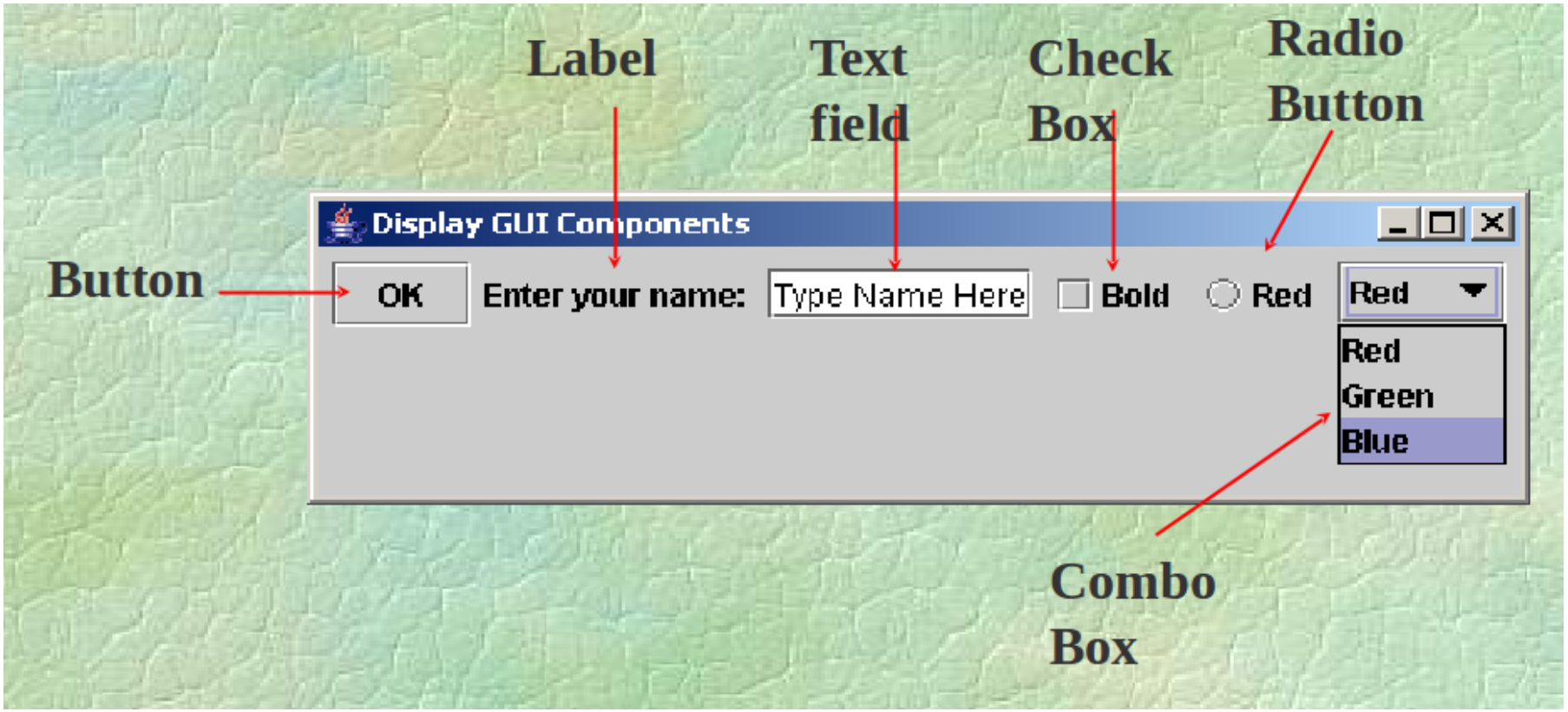
Exemple avec Swing



Exemple avec Swing

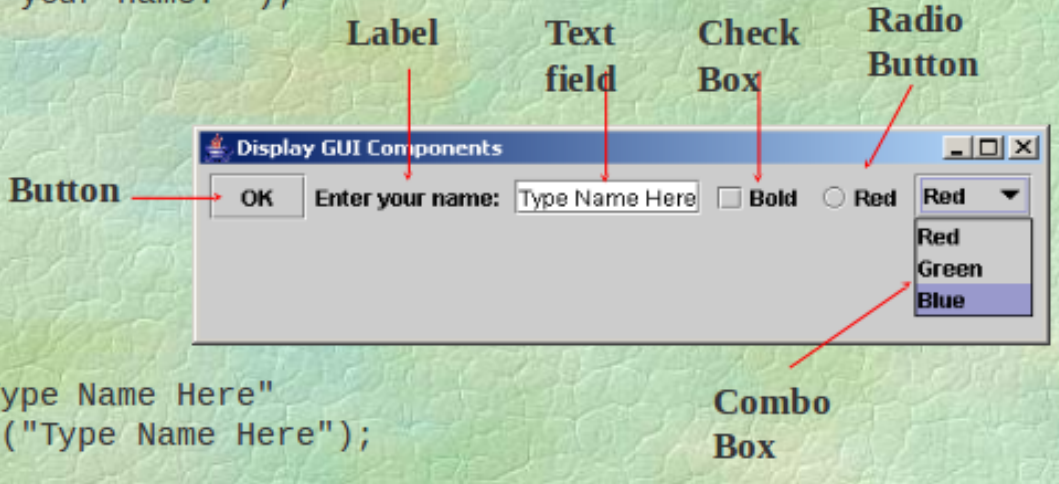
Éxecution du programme *umts1.java*

Un exemple "graphique"



```
// Create a button with text OK
JButton jbtOK = new JButton("OK");

// Create a label with text "Enter your name: "
JLabel jlblName = new JLabel("Enter your name: ");
```




```
// Create a text field with text "Type Name Here"
JTextField jtfName = new JTextField("Type Name Here");

// Create a check box with text bold
JCheckBox jchkBold = new JCheckBox("Bold");

// Create a radio button with text red
JRadioButton jrbRed = new JRadioButton("Red");

// Create a combo box with choices red, green, and blue
JComboBox jcboColor = new JComboBox(new String[]{"Red", "Green", "Blue"});
```

NetBeans

- `NetBeans` constitue une plate forme qui permet le développement d'applications spécifiques (bibliothèque Swing) ;
- Dans ce qui suit, on s'intèresse à cet IDE (environnement de développement intégré) ;
- Pour installer `NetBeans` avec `JDK`, vous pouvez consulter :

  `http://www.oracle.com/technetwork/java/javase/downloads/index.html`

  et choisissez `NetBeans with JDK 8`.

NetBeans

- `NetBeans` constitue une plate forme qui permet le développement d'applications spécifiques (bibliothèque Swing) ;
- Dans ce qui suit, on s'intèresse à cet IDE (environnement de développement intégré) ;
- Pour installer `NetBeans` avec `JDK`, vous pouvez consulter :

  `http://www.oracle.com/technetwork/java/javase/downloads/index.html`

  et choisissez `NetBeans with JDK 8`.

NetBeans

- `NetBeans` constitue une plate forme qui permet le développement d'applications spécifiques (bibliothèque Swing) ;
- Dans ce qui suit, on s'intèresse à cet IDE (environnement de développement intégré) ;
- Pour installer `NetBeans` avec `JDK`, vous pouvez consulter :

  `http://www.oracle.com/technetwork/java/javase/downloads/index.html`

  et choisissez `NetBeans with JDK 8`.

Création d'un projet

- Créez un nouveau projet en cliquant `File + New Project + Java + Java Application` puis en entrant le nom du projet ;
- Notre projet initial se compose uniquement d'un fichier `Main.java`. Il s'agit par défaut d'une application console

Création d'un projet

- Créez un nouveau projet en cliquant `File + New Project + Java + Java Application` puis en entrant le nom du projet ;
- Notre projet initial se compose uniquement d'un fichier `Main.java`. Il s'agit par défaut d'une application console

Création d'un projet

JavaApplication1 - NetBeans IDE 8.2

File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help

<default config>

Projects × Files Services

JavaApplication1

Navigator ×

Members <empty>

JavaApplication1
main(String[] args)

JavaApplication1.java ×

Source History

```
1  /*
2   * To change this license header, choose License Headers in Project Properties.
3   * To change this template file, choose Tools | Templates
4   * and open the template in the editor.
5   */
6  package javaapplication1;
7
8  /**
9   *
10  * @author gabli
11  */
12 public class JavaApplication1 {
13
14     /**
15      * @param args the command line arguments
16      */
17     public static void main(String[] args) {
18         // TODO code application logic here
19     }
20
21 }
22
```

Ajouter un composant visuel au projet

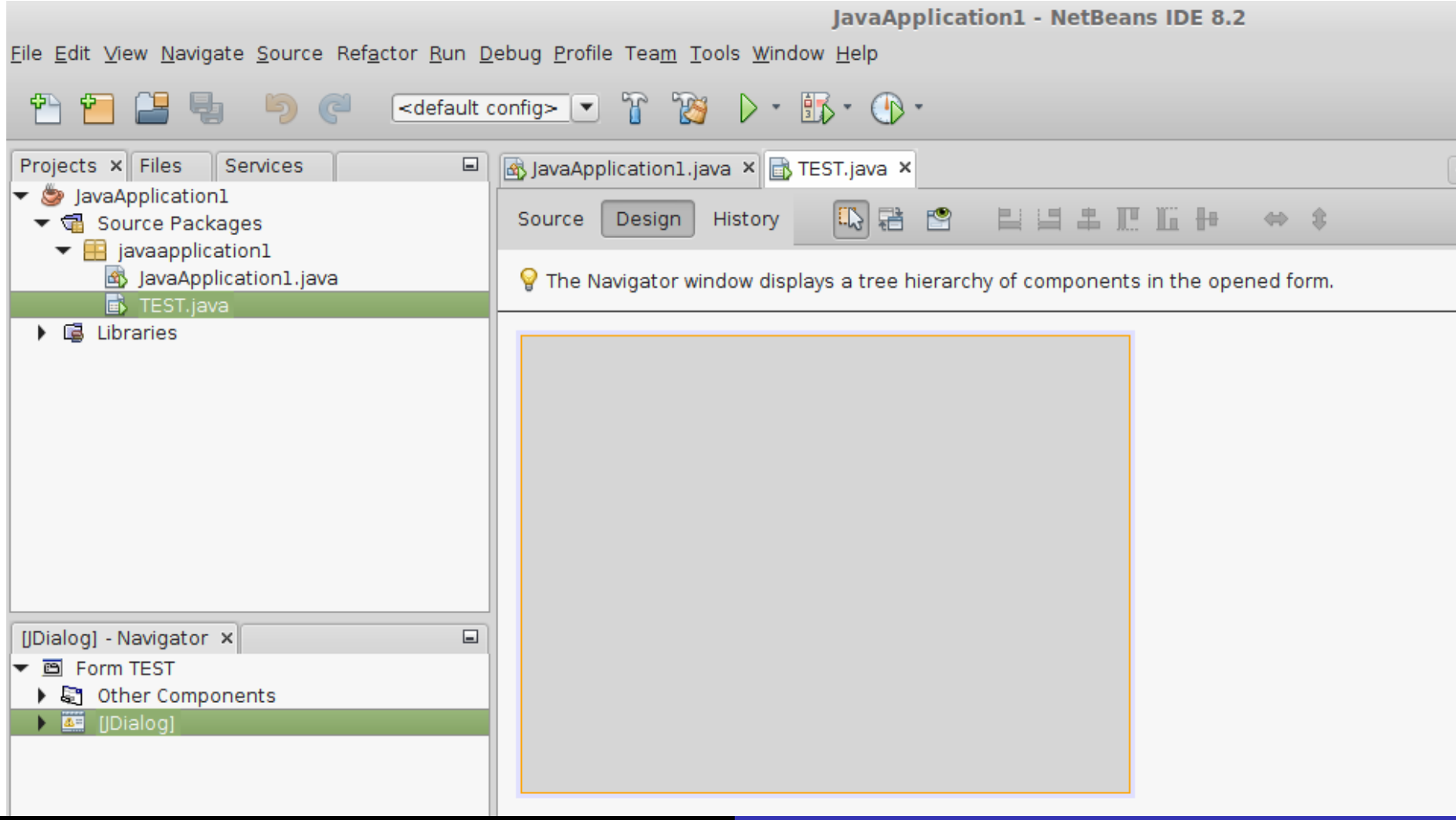
- Il y a de nombreux types différents de composants visuels. Nous utiliserons ici un composant simple : `JDialog Form`;
- Pour pouvoir faire de design, on doit procéder comme suit : `File + New File + Swing GUI Forms + JDialog Form` puis on choisit le nom de notre fichier, par exemple `Test.java`;
- Le projet en cours est mis à jour et possède maintenant deux fichiers : JavaApplication1.java et Test.Java

Ajouter un composant visuel au projet

- Il y a de nombreux types différents de composants visuels. Nous utiliserons ici un composant simple : `JDialog Form`;
- Pour pouvoir faire de design, on doit procéder comme suit : `File + New File + Swing GUI Forms + JDialog Form` puis on choisit le nom de notre fichier, par exemple `Test.java`;
- Le projet en cours est mis à jour et possède maintenant deux fichiers : JavaApplication1.java et Test.Java

Ajouter un composant visuel au projet

- Il y a de nombreux types différents de composants visuels. Nous utiliserons ici un composant simple : `JDialog Form`;
- Pour pouvoir faire de design, on doit procéder comme suit : `File + New File + Swing GUI Forms + JDialog Form` puis on choisit le nom de notre fichier, par exemple `Test.java`;
- Le projet en cours est mis à jour et possède maintenant deux fichiers : JavaApplication1.java et Test.Java


JavaApplication1 - NetBeans IDE 8.2
File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help
<default config>
Projects
Files
Services
JavaApplication1
Source Packages
javaapplication1
JavaApplication1.java
TEST.java
Libraries
JavaApplication1.java
TEST.java
Source
Design
History
The Navigator window displays a tree hierarchy of components in the opened form.
[JDialog] - Navigator
Form TEST
Other Components
[JDialog]

Suppression d'un fichier du projet

Souhaitons maintenant supprimer le fichier `JavaApplication1.java`.

Pour ceci, on fait un click droit sur le fichier `JavaApplication1`.java, et on choisit `Delete` dans le menu contextuel.

Compilation du projet

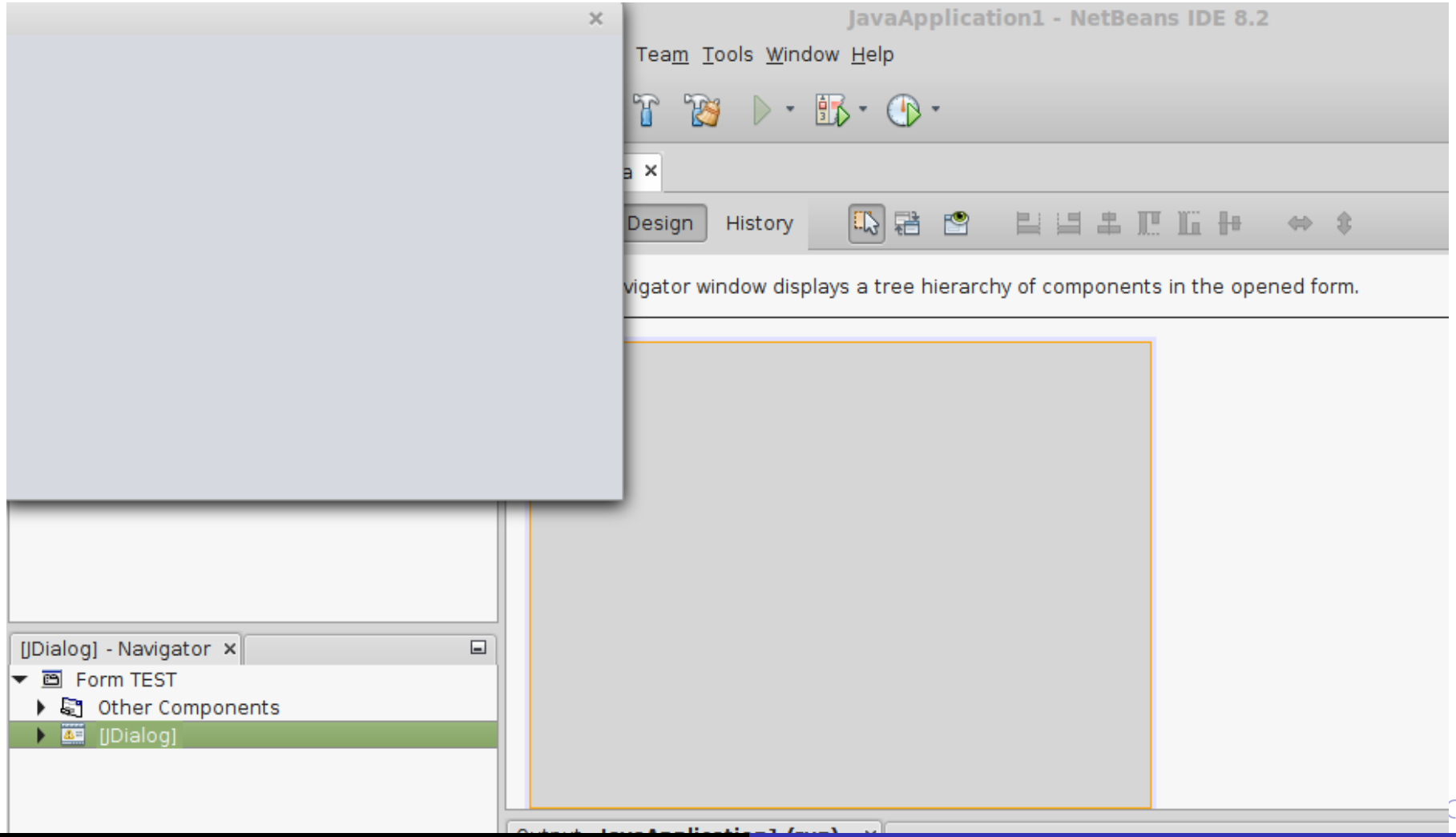
- On clique sur le bouton `Run Project` ;
- NetBeans vous demandera de sélectionner la classe principale ;
- En validant `Test` comme étant la classe principale, on obtient comme résultat une fenêtre vide (sans nom, sans bouton....).

Compilation du projet

- On clique sur le bouton `Run Project` ;
- NetBeans vous demandera de sélectionner la classe principale ;
- En validant `Test` comme étant la classe principale, on obtient comme résultat une fenêtre vide (sans nom, sans bouton....).

Compilation du projet

- On clique sur le bouton `Run Project` ;
- NetBeans vous demandera de sélectionner la classe principale ;
- En validant `Test` comme étant la classe principale, on obtient comme résultat une fenêtre vide (sans nom, sans bouton....).


JavaApplication1 - NetBeans IDE 8.2
Team Tools Window Help
Design History
vigator window displays a tree hierarchy of components in the opened form.
[JDialog] - Navigator
Form TEST
Other Components
[JDialog]

Éléments de base d'une fenêtre

- Une fenêtre graphique peut se manipuler sous forme graphique (l'onglet `Design`) ou sous forme textuelle (l'onglet `Source`) ;
- La construction d'une interface se fait en mode `Design`.
- Tous les objets graphiques nécessaires à une interface sont regroupés dans l'onglet `Palette` ;
- Pour faire le design, il suffit de cliquer sur l'objet voulu (bouton par exemple) et sans lacher la souris on met l'objet sur la partie design.

Éléments de base d'une fenêtre

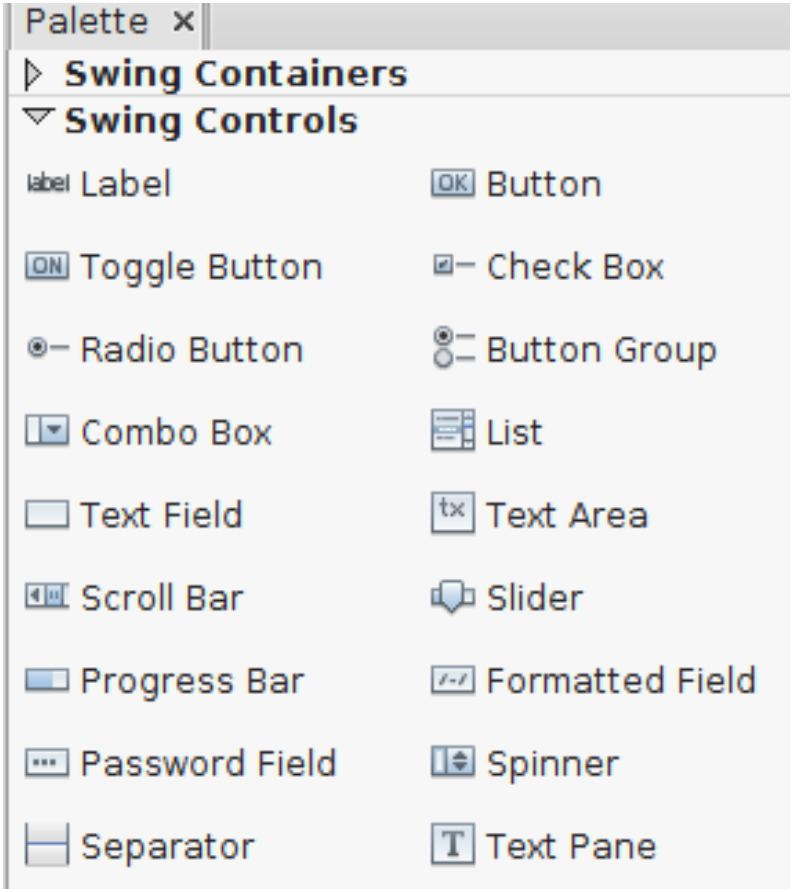
- Une fenêtre graphique peut se manipuler sous forme graphique (l'onglet `Design`) ou sous forme textuelle (l'onglet `Source`) ;
- La construction d'une interface se fait en mode `Design`.
- Tous les objets graphiques nécessaires à une interface sont regroupés dans l'onglet `Palette` ;
- Pour faire le design, il suffit de cliquer sur l'objet voulu (bouton par exemple) et sans lacher la souris on met l'objet sur la partie design.

Éléments de base d'une fenêtre

- Une fenêtre graphique peut se manipuler sous forme graphique (l'onglet `Design`) ou sous forme textuelle (l'onglet `Source`) ;
- La construction d'une interface se fait en mode `Design`.
- Tous les objets graphiques nécessaires à une interface sont regroupés dans l'onglet `Palette` ;
- Pour faire le design, il suffit de cliquer sur l'objet voulu (bouton par exemple) et sans lacher la souris on met l'objet sur la partie design.

Éléments de base d'une fenêtre

- Une fenêtre graphique peut se manipuler sous forme graphique (l'onglet `Design`) ou sous forme textuelle (l'onglet `Source`) ;
- La construction d'une interface se fait en mode `Design`.
- Tous les objets graphiques nécessaires à une interface sont regroupés dans l'onglet `Palette` ;
- Pour faire le design, il suffit de cliquer sur l'objet voulu (bouton par exemple) et sans lacher la souris on met l'objet sur la partie design.


Palette
Swing Containers
Swing Controls
Label
Button
Toggle Button
Check Box
Radio Button
Button Group
Combo Box
List
Text Field
Text Area
Scroll Bar
Slider
Progress Bar
Formatted Field
Password Field
Spinner
Separator
Text Pane

- Les évènements sont des actions déclenchées par l'utilisateur en utilisant l'interface graphique, par exemple : taper au clavier, appuyer sur un bouton, redimensionner une fenêtre ...
- En Java, chaque source d'évènement (le bouton appuyé par exemple) envoie un message à un objet qui va gérer cet évènement ;
- Cet objet est nommé `écouteur d'évènement (event listener)` ;
- La classe ancêtre gérant les événements est `EventObject`.
- Il existe des sous classes qui s'occupent d'évènements particuliers comme `ActionEvent` (évènements des boutons, ...), `WindowEvent` (évènements des fenêtres), ...

- Les évènements sont des actions déclenchées par l'utilisateur en utilisant l'interface graphique, par exemple : taper au clavier, appuyer sur un bouton, redimensionner une fenêtre ...
- En Java, chaque source d'évènement (le bouton appuyé par exemple) envoie un message à un objet qui va gérer cet évènement ;
- Cet objet est nommé `écouteur d'évènement (event listener)` ;
- La classe ancêtre gérant les événements est `EventObject`.
- Il existe des sous classes qui s'occupent d'évènements particuliers comme `ActionEvent` (évènements des boutons, ...), `WindowEvent` (évènements des fenêtres), ...

- Les évènements sont des actions déclenchées par l'utilisateur en utilisant l'interface graphique, par exemple : taper au clavier, appuyer sur un bouton, redimensionner une fenêtre ...
- En Java, chaque source d'évènement (le bouton appuyé par exemple) envoie un message à un objet qui va gérer cet évènement ;
- Cet objet est nommé `écouteur d'évènement (event listener)` ;
- La classe ancêtre gérant les événements est `EventObject`.
- Il existe des sous classes qui s'occupent d'évènements particuliers comme `ActionEvent` (évènements des boutons, ...), `WindowEvent` (évènements des fenêtres), ...

- Les évènements sont des actions déclenchées par l'utilisateur en utilisant l'interface graphique, par exemple : taper au clavier, appuyer sur un bouton, redimensionner une fenêtre ...
- En Java, chaque source d'évènement (le bouton appuyé par exemple) envoie un message à un objet qui va gérer cet évènement ;
- Cet objet est nommé `écouteur d'évènement (event listener)` ;
- La classe ancêtre gérant les événements est `EventObject`.
- Il existe des sous classes qui s'occupent d'évènements particuliers comme `ActionEvent` (évènements des boutons, ...), `WindowEvent` (évènements des fenêtres), ...

- Les évènements sont des actions déclenchées par l'utilisateur en utilisant l'interface graphique, par exemple : taper au clavier, appuyer sur un bouton, redimensionner une fenêtre ...
- En Java, chaque source d'évènement (le bouton appuyé par exemple) envoie un message à un objet qui va gérer cet évènement ;
- Cet objet est nommé `écouteur d'évènement (event listener)` ;
- La classe ancêtre gérant les événements est `EventObject`.
- Il existe des sous classes qui s'occupent d'évènements particuliers comme `ActionEvent` (évènements des boutons, ...), `WindowEvent` (évènements des fenêtres), ...

Exemple

- Supposons qu'on veut afficher un message dans un `jTextArea` après un `clic sur le bouton`;
- Pour ceci, on développe deux partie : une partie `Design` et une partie `Source`.

Le design :

- dans la palette on choisit un bouton (`Button`) et une zone de texte (`TextArea`).
- Si on veut changer le label du bouton, on sélectionne le bouton, et dans la partie propriété (`Properties`) on choisit la propriété `Text`, et on met le label choisi, par exemple : "Afficher".

Exemple

- Supposons qu'on veut afficher un message dans un `jTextArea` après un `clic sur le bouton`;
- Pour ceci, on développe deux partie : une partie `Design` et une partie `Source`.

Le design :

- dans la palette on choisit un bouton (`Button`) et une zone de texte (`TextArea`).
- Si on veut changer le label du bouton, on sélectionne le bouton, et dans la partie propriété (`Properties`) on choisit la propriété `Text`, et on met le label choisi, par exemple : "Afficher".

Exemple

- Supposons qu'on veut afficher un message dans un `jTextArea` après un `clic sur le bouton`;
- Pour ceci, on développe deux partie : une partie `Design` et une partie `Source`.

Le design :

- dans la palette on choisit un bouton (`Button`) et une zone de texte (`TextArea`).
- Si on veut changer le label du bouton, on sélectionne le bouton, et dans la partie propriété (`Properties`) on choisit la propriété `Text`, et on met le label choisi, par exemple : "Afficher".

Exemple

- Supposons qu'on veut afficher un message dans un `jTextArea` après un `clic sur le bouton`;
- Pour ceci, on développe deux partie : une partie `Design` et une partie `Source`.

Le design :

- dans la palette on choisit un bouton (`Button`) et une zone de texte (`TextArea`).
- Si on veut changer le label du bouton, on sélectionne le bouton, et dans la partie propriété (`Properties`) on choisit la propriété `Text`, et on met le label choisi, par exemple : "Afficher".

Exemple

La source :
On fait d'abord un double clic sur le bouton, puis on entre le code suivant :

```
private void jButton1ActionPerformed(java.
    awt.event.ActionEvent evt) {
        jTextArea1.setText("Bonjour") ;
    }
```

Après la compilation et si on fait un clic sur le bouton "Afficher", on obtient le message "Bonjour" dans la zone de texte.

Exemple

La source :
On fait d'abord un double clic sur le bouton, puis on entre le code suivant :

```
private void jButton1ActionPerformed(java.
    awt.event.ActionEvent evt) {
        jTextArea1.setText("Bonjour") ;
    }
```

Après la compilation et si on fait un clic sur le bouton "Afficher", on obtient le message "Bonjour" dans la zone de texte.

Exemple 2

Dans cet exemple on veut faire la somme des deux entiers entrés par l'utilisateur.

Le design :

On utilise deux TextFields, un TextArea et un Button.

La source :

```
private void jButton1ActionPerformed(java.awt.event.
    ActionEvent evt) {
    int a = Integer.parseInt(jTextField1.getText())+
            Integer.parseInt(jTextField2.getText());
    String b = String.valueOf(a);
    jTextArea1.setText(b);
  }
```

Exemple 2

Dans cet exemple on veut faire la somme des deux entiers entrés par l'utilisateur.

Le design :

On utilise deux TextFields, un TextArea et un Button.

La source :

```
private void jButton1ActionPerformed(java.awt.event.
    ActionEvent evt) {
    int a = Integer.parseInt(jTextField1.getText())+
            Integer.parseInt(jTextField2.getText());
    String b = String.valueOf(a);
    jTextArea1.setText(b);
  }
```

Exemple 2

Dans cet exemple on veut faire la somme des deux entiers entrés par l'utilisateur.

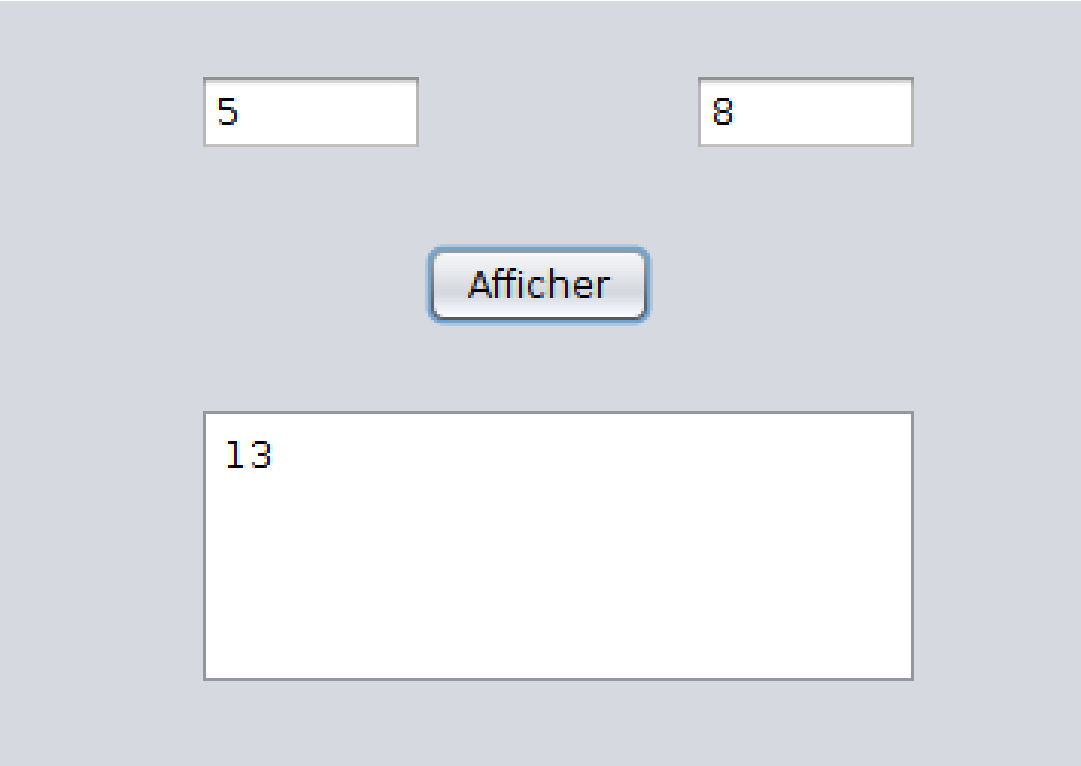
Le design :

On utilise deux TextFields, un TextArea et un Button.

La source :

```
private void jButton1ActionPerformed(java.awt.event.
    ActionEvent evt) {
    int a = Integer.parseInt(jTextField1.getText())+
            Integer.parseInt(jTextField2.getText());
    String b = String.valueOf(a);
    jTextArea1.setText(b);
  }
```

Exécution



Exemple 3

Lire à partir d'un fichier.

Le design :

On utilise un TextArea et un Button.

La source :

```
private void jButton1ActionPerformed(java.awt.event.
    ActionEvent evt){
     String ligne;
     try{
       BufferedReader b = new BufferedReader(new FileReader("
           Test.txt"));
       while((ligne=b.readLine())!=null){
           jTextArea1.append(ligne+"\r\n");
       }
       b.close();
   }catch(IOException e){
       System.out.println("Erreur de lire le fichier!");
   }
```

Exemple 3

Lire à partir d'un fichier.

Le design :

On utilise un TextArea et un Button.

La source :

```
private void jButton1ActionPerformed(java.awt.event.
    ActionEvent evt){
     String ligne;
     try{
       BufferedReader b = new BufferedReader(new FileReader("
           Test.txt"));
       while((ligne=b.readLine())!=null){
           jTextArea1.append(ligne+"\r\n");
       }
       b.close();
   }catch(IOException e){
       System.out.println("Erreur de lire le fichier!");
   }
```

Exemple 3

Lire à partir d'un fichier.

Le design :

On utilise un TextArea et un Button.

La source :

```
private void jButton1ActionPerformed(java.awt.event.
    ActionEvent evt){
     String ligne;
     try{
       BufferedReader b = new BufferedReader(new FileReader("
           Test.txt"));
       while((ligne=b.readLine())!=null){
           jTextArea1.append(ligne+"\r\n");
       }
       b.close();
   }catch(IOException e){
       System.out.println("Erreur de lire le fichier!");
   }
```

Les cases à cocher

Les cases à cocher

La case à cocher de type JCheckBox permet à l'utilisateur d'effectuer un choix de type oui ou non.

```
import java.awt.* ; import java.awt.event.* ;
import javax.swing.* ; import javax.swing.event.* ;
class MaFenetre extends JFrame{
    private JCheckBox MaCase;
    public MaFenetre (){
      super("Une fenetre avec une case");
      setBounds(10,40,300,200);
      MaCase = new JCheckBox("Une case");
      //getContentPane().add(MaCase);
      add(MaCase); // a partir de la version java5
   }
}
```

```
public class TestCaseAcocher {
  public static void main(String[] args) {
    JFrame fen = new MaFenetre();
    fen.setVisible(true);
    fen.setDefaultCloseOperation(JFrame.
        EXIT_ON_CLOSE);
    }
```

Les cases à cocher

- Par défaut, une case à cocher est construite dans l'état non coché (false) ;
- La méthode isSelected de la classe AbstractButton permet de connaître l'état (true ou false) d'une case à cocher à un moment donné ;
- La méthode setSelected de la classe AbstractButton permet de modifier l'état d'une case à cocher.

Exemple :

```
MaCase.setSelected(true) ;
//coche la case de référence MaCase
```

Exemple

1. Dessiner les radio boutons et les checkboxs comme indiqué dans la figure 1.
2. lorsqu'on clique sur le bouton :
   - la fenêtre prend la couleur de la case cochée ;
   - Dans le textArea on affiche le résultat du produit des checkboxs cochés et du radiobutton coché.

Exemple

FIGURE – Design de l'exercice 1.

Choisir un groupe

Remarque :

Dans le design et pour qu'on puisse sélectionner une seule radio button et une seule checkbox, il faut ajouter toutes les radio buttons à un seul groupe, et toutes les checkboxs à un autre groupe.

Pour ce faire, dans Properties-buttonGroup de chaque radio button et chaque checkbox, on doit choisir le groupe convenable.

Exemple

Rouge
4
1
Bleu
5
2
Jaune
6
3
Afficher
Combo Box
List
Text Field
Text Area
Scroll Bar
Slider
jRadioButton4 [JRadioButton] - Propert...
Properties
Binding
Events
Code
model
<default>
buttonGroup
buttonGroup1
font
DejaVu Sans 12 Plair
jRadioButton4 [JRadioButton]

Exemple

```
private void jButton1ActionPerformed(java.awt.event.ActionEvent
      evt) {
        panel2.add(jRadioButton4);
        panel2.add(jRadioButton5);
        panel2.add(jRadioButton6);
        if(jRadioButton4.isSelected())
            panel2.setBackground(Color.red);
        else if(jRadioButton5.isSelected())
            panel2.setBackground(Color.blue);
        else if(jRadioButton6.isSelected())
            panel2.setBackground(Color.yellow);
```

Exemple

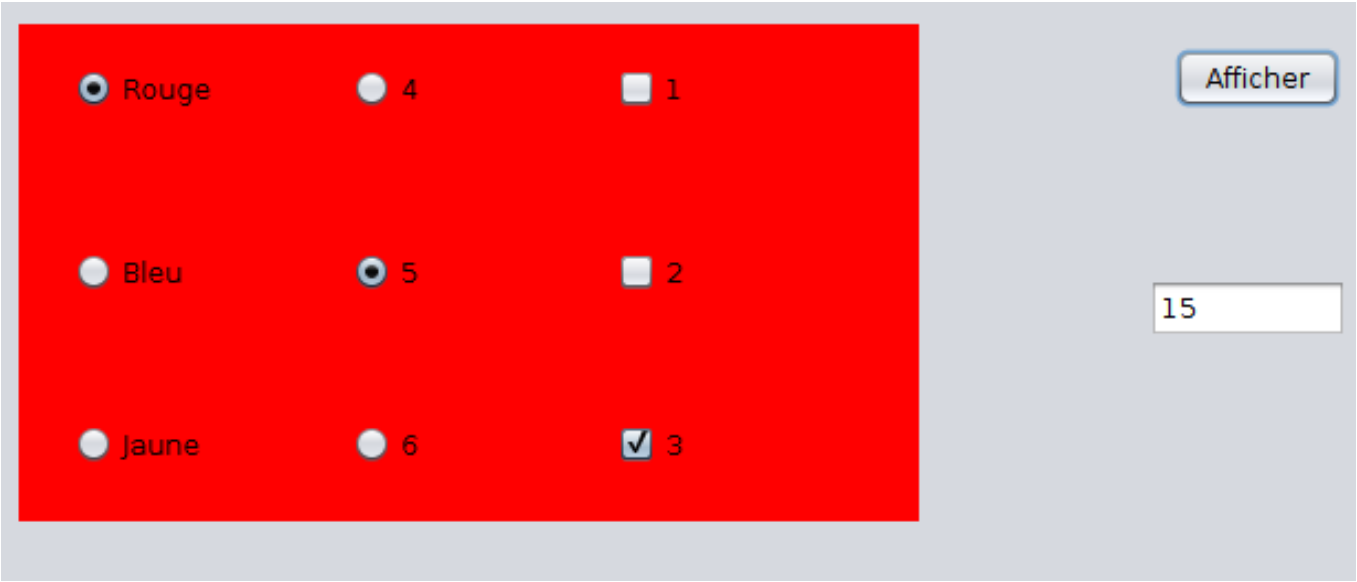
```
int a = 0, b = 0;
if(jRadioButton1.isSelected())
    a = Integer.parseInt(jRadioButton1.getText ( ) );
else if(jRadioButton2.isSelected())
    a = Integer.parseInt(jRadioButton2.getText ( ) );
else if(jRadioButton3.isSelected())
    a = Integer.parseInt(jRadioButton3.getText ( ) );
```

Exemple

```
if(jCheckBox1.isSelected())
        b = Integer.parseInt(jCheckBox1.getText ( ) );
else if(jCheckBox2.isSelected())
        b = Integer.parseInt(jCheckBox2.getText ( ) );
else if(jCheckBox3.isSelected())
        b = Integer.parseInt(jCheckBox3.getText ( ) );

jTextField1.setText(String.valueOf(a*b));
    }
```

Exemple



Exercice

Réalisez l'exemple précédent

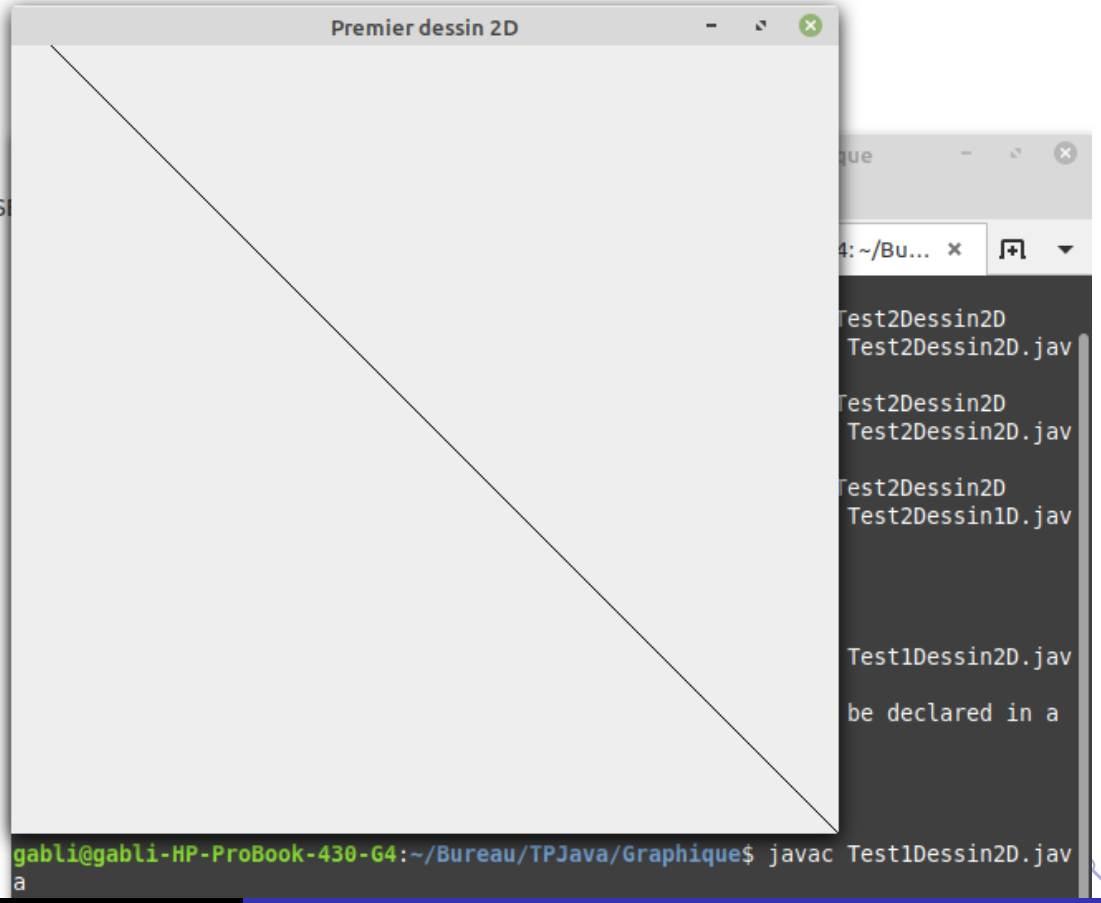
Graphique 2D

Dessiner une droite dans la fenêtre

```
 import java.awt.*;
import javax.swing.*;
class MaFenetre extends JFrame{
    public MaFenetre(){
      setTitle("Premier dessin 2D");
      setSize(500,500);
      setVisible(true);
      setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
      }
     public void paint(Graphics g){
      Graphics2D g2D = (Graphics2D) g;
      g2D.drawLine(0, 0, 500, 500);
      }
}
public class Test1Dessin2D{
    public static void main(String[] args){
      MaFenetre F = new MaFenetre();
      }
}
```

Exemple

```
import java.awt.*;
import javax.swing.*;
class MaFenetre extends JFrame{
    public MaFenetre() {
        setTitle("Premier dessin 2D");
        setSize(500,500);
        setVisible(true);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOS
    }
    public void paint(Graphics g) {
        Graphics2D g2D = (Graphics2D) g;
        g2D.drawLine(0, 0, 500, 500);
    }
}
public class Test1Dessin2D {
    public static void main(String[] args) {
        MaFenetre F = new MaFenetre();
    }
}
```



Remarques

- **Graphics2D** est une sous classe de **Graphics** ;
- La méthode **paint** reçoit en paramètre un objet de type **Graphics** mais la classe réelle de ce contexte graphique est toujours **Graphics2D**. Afin de pouvoir utiliser toutes les possibilités du graphisme de Java 2, il faut le convertir en un objet de type **Graphics2D**.
- Dans le programme précédent, on a essayé de dessiner la droite d'origine $(0, 0)$ directement dans la fenêtre, donc il faut voir l'extention de la droite pour atteindre le point $(0, 0)$ de la fenêtre.
- Pour faire mieu, on dessine la droite dans un conteneur tel que un panneau (jPanel).

Remarques

- **Graphics2D** est une sous classe de **Graphics** ;
- La méthode **paint** reçoit en paramètre un objet de type **Graphics** mais la classe réelle de ce contexte graphique est toujours **Graphics2D**. Afin de pouvoir utiliser toutes les possibilités du graphisme de Java 2, il faut le convertir en un objet de type **Graphics2D**.
- Dans le programme précédent, on a essayé de dessiner la droite d'origine $(0, 0)$ directement dans la fenêtre, donc il faut voir l'extention de la droite pour atteindre le point $(0, 0)$ de la fenêtre.
- Pour faire mieu, on dessine la droite dans un conteneur tel que un panneau (jPanel).

Remarques

- **Graphics2D** est une sous classe de **Graphics** ;
- La méthode **paint** reçoit en paramètre un objet de type **Graphics** mais la classe réelle de ce contexte graphique est toujours **Graphics2D**. Afin de pouvoir utiliser toutes les possibilités du graphisme de Java 2, il faut le convertir en un objet de type **Graphics2D**.
- Dans le programme précédent, on a essayé de dessiner la droite d'origine $(0, 0)$ directement dans la fenêtre, donc il faut voir l'extention de la droite pour atteindre le point $(0, 0)$ de la fenêtre.
- Pour faire mieu, on dessine la droite dans un conteneur tel que un panneau (jPanel).

Remarques

- **Graphics2D** est une sous classe de **Graphics** ;
- La méthode **paint** reçoit en paramètre un objet de type **Graphics** mais la classe réelle de ce contexte graphique est toujours **Graphics2D**. Afin de pouvoir utiliser toutes les possibilités du graphisme de Java 2, il faut le convertir en un objet de type **Graphics2D**.
- Dans le programme précédent, on a essayé de dessiner la droite d'origine $(0, 0)$ directement dans la fenêtre, donc il faut voir l'extention de la droite pour atteindre le point $(0, 0)$ de la fenêtre.
- Pour faire mieu, on dessine la droite dans un conteneur tel que un panneau (jPanel).

Dessiner une droite dans un panneau

```
import java.awt.*;import javax.swing.*;
//ajout d'un Jpanel
class MonPanneau extends JPanel{
    public MonPanneau(){
        this.setPreferredSize(new Dimension(500,500));
        }
    public void paint(Graphics g){//paint est maintenant dans le
         panneau
      Graphics2D g2D = (Graphics2D) g;
      g2D.drawLine(0, 0, 500, 500);
      }
}
```

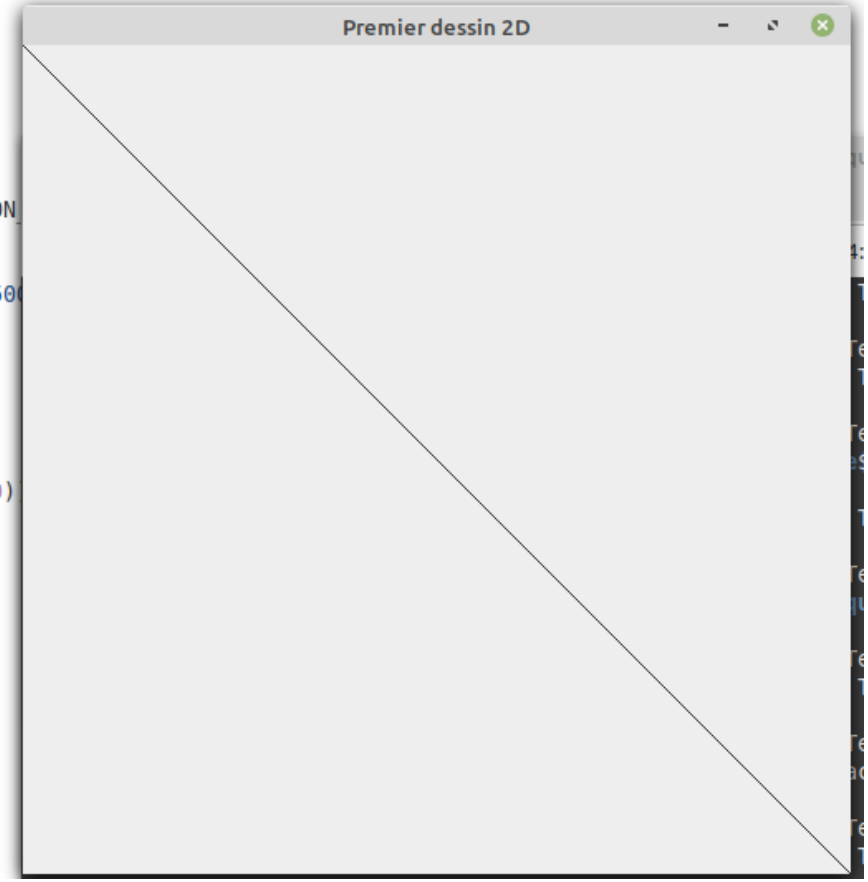
Dessiner une droite dans un panneau

```
class MaFenetre extends JFrame{
    MonPanneau panel = new MonPanneau();
    public MaFenetre(){
      this.setTitle("Premier dessin 2D");
      this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
      this.add(panel);
      this.pack();//definit la taille de la fenetre en fonction
          des tailles preferees de son contenu
      this.setLocationRelativeTo(null);//centrer l'application
      this.setVisible(true);
      }
}
public class Test2Dessin2D{
    public static void main(String[] args){
      MaFenetre F = new MaFenetre();
      }
}
```

Exemple

```
import java.awt.*;
import javax.swing.*;
//ajout d'un Jpanel
class MaFenetre extends JFrame{
    MonPanneau panel = new MonPanneau();;
    public MaFenetre(){
        this.setTitle("Premier dessin 2D");
        this.setDefaultCloseOperation(JFrame.EXIT_ON
        //panel = new MonPanneau();
        this.add(panel);
        this.pack();//au lieu de this.setSize(500,50
        this.setLocationRelativeTo(null);
        this.setVisible(true);
    }
}
class MonPanneau extends JPanel{
    public MonPanneau(){
        this.setPreferredSize(new Dimension(500,500)
    }
    public void paint(Graphics g) {
        Graphics2D g2D = (Graphics2D) g;
        g2D.drawLine(0, 0, 500, 500);
    }
}

public class Test2Dessin2D{
    public static void main(String[] args) {
        MaFenetre F = new MaFenetre();
    }
}
```



Remarque

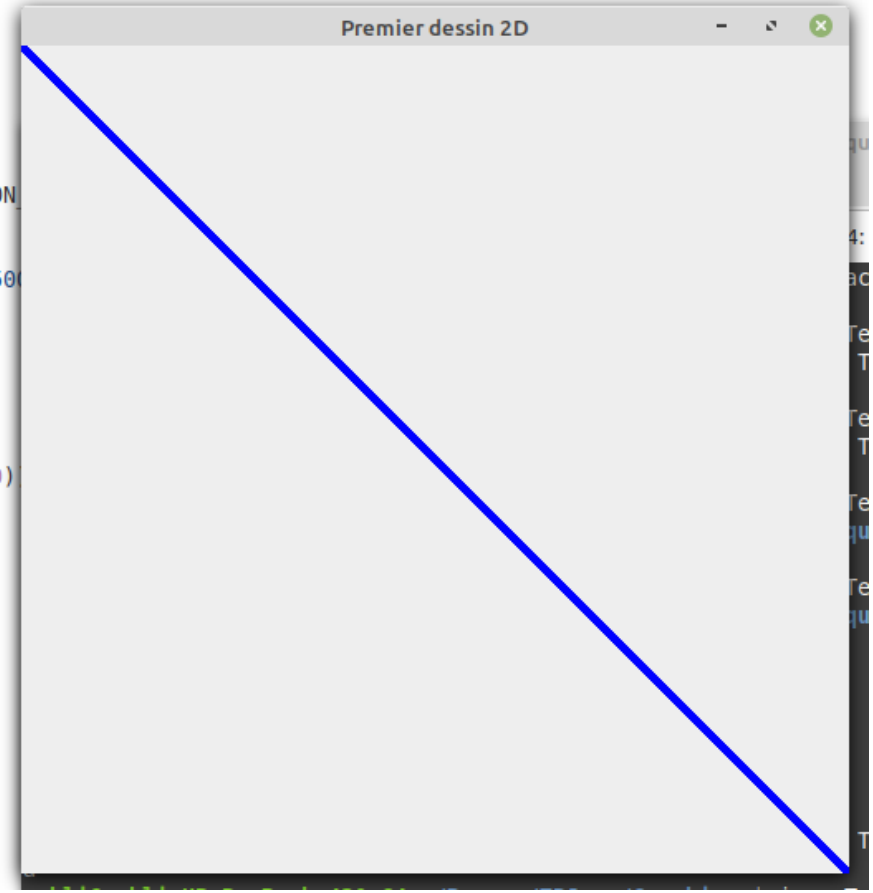
Il faut remarquer que la méthode `paint` est maintenant dans la classe `MonPanneau`.
Pour modifier la coleur et le fond de la droite, on ajoute les deux instructions
g2D.setStroke(new BasicStroke(5)) ;
g2D.setPaint(Color.blue) ;

Dessiner une droite dans un panneau

```
import java.awt.*;import javax.swing.*;
//ajout d'un Jpanel
class MonPanneau extends JPanel{
    public MonPanneau(){
        this.setPreferredSize(new Dimension(500,500));
        }
    public void paint(Graphics g){//paint est maintenant dans le
         panneau
      Graphics2D g2D = (Graphics2D) g;
      g2D.setStroke(new BasicStroke(5));
      g2D.setPaint(Color.blue);
      g2D.drawLine(0, 0, 500, 500);
      }
}
```

```
import java.awt.*;
import javax.swing.*;
//ajout d'un Jpanel
class MaFenetre extends JFrame{
    MonPanneau panel = new MonPanneau();
    public MaFenetre(){
        this.setTitle("Premier dessin 2D");
        this.setDefaultCloseOperation(JFrame.EXIT_ON
        //panel = new MonPanneau();
        this.add(panel);
        this.pack();//au lieu de this.setSize(500,50
        this.setLocationRelativeTo(null);
        this.setVisible(true);
    }
}
class MonPanneau extends JPanel{
    public MonPanneau(){
        this.setPreferredSize(new Dimension(500,500)
    }
    public void paint(Graphics g) {
        Graphics2D g2D = (Graphics2D) g;
        g2D.setStroke(new BasicStroke(5));
        g2D.setPaint(Color.blue);
        g2D.drawLine(0, 0, 500, 500);
    }
}

public class Test2Dessin2D{
    public static void main(String[] args) {
        MaFenetre F = new MaFenetre();
    }
}
```



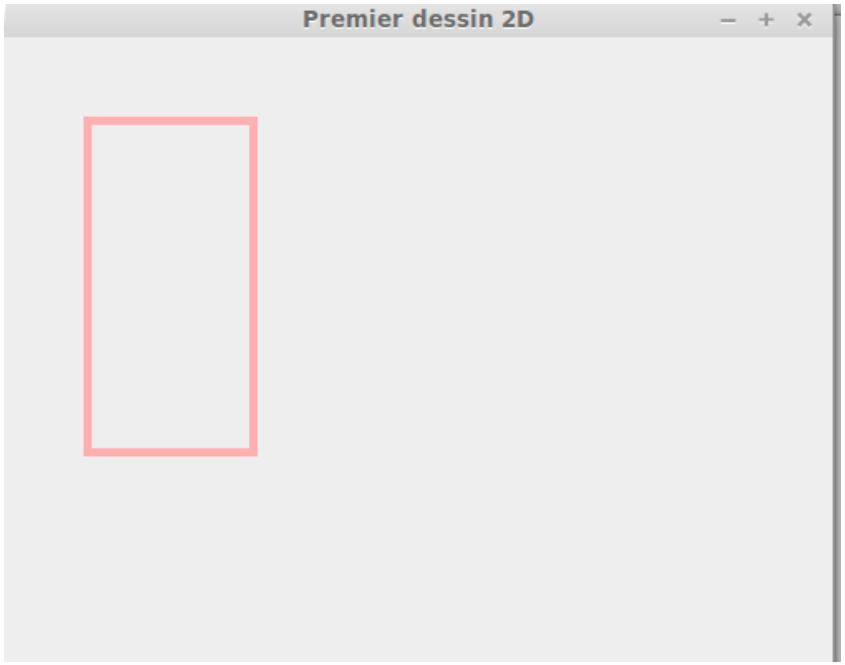
Dessiner un rectangle

```
import java.awt.*;import javax.swing.*;
class MaFenetre extends JFrame{
    MonPanneau panel = new MonPanneau();
    public MaFenetre(){
      this.setTitle("Premier dessin 2D");
      this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
      this.add(panel);
      this.pack();//au lieu de this.setSize(500,500);
      this.setLocationRelativeTo(null);
      this.setVisible(true);
      }
  }
```

Dessiner un rectangle

```
class MonPanneau extends JPanel{
    public MonPanneau(){
        this.setPreferredSize(new Dimension(500,500));
        }
    public void paint(Graphics g){//paint est maintenant dans le
         panneau
      Graphics2D g2D = (Graphics2D) g;
      g2D.setStroke(new BasicStroke(5));
      g2D.setPaint(Color.pink);
      g2D.drawRect(50, 50, 100, 200);
      }
  }
public class DessinerRectangle{
    public static void main(String[] args){
      MaFenetre F = new MaFenetre();
      }
  }
```

Exemple

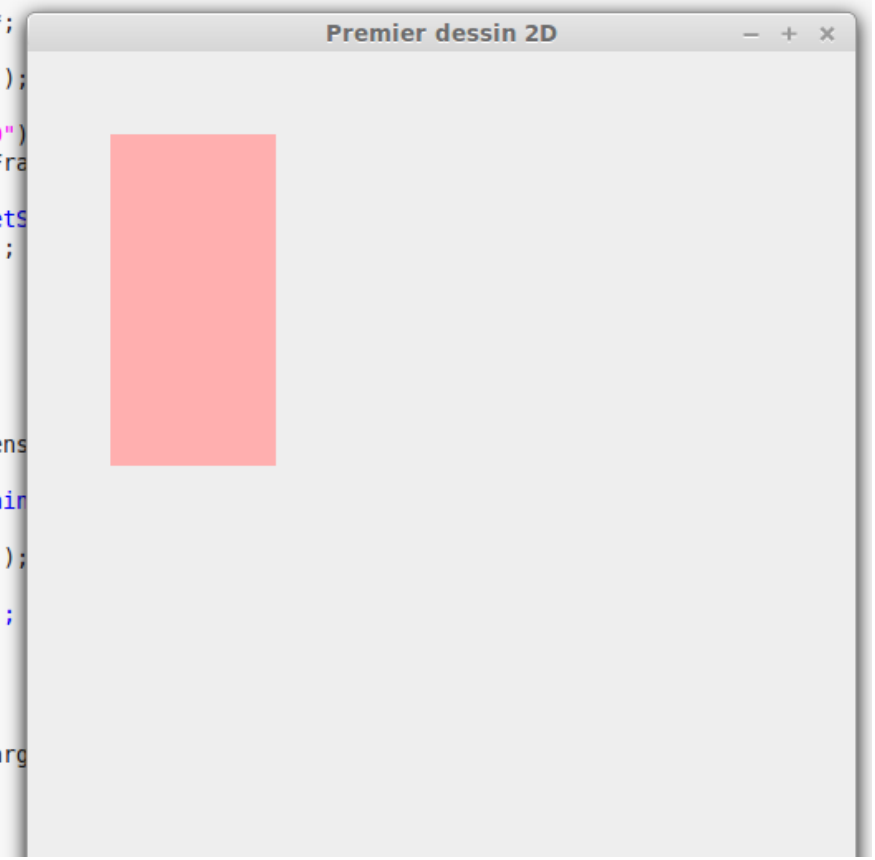


Remarque

- Dans l'instruction `g2D.drawRect(50, 50, 100, 200);` le point $(50, 50)$ est le coté supérieur du rectangle, 100 est sa largeur et 200 sa longueur ;
- Si on veut dessiner aussi l'intérieur du rectangle, on utilise l'instruction `g2D.fillRect(50, 50, 100, 200);`

Exemple

```
import java.awt.*;import javax.swing.*;
class MaFenetre extends JFrame{
    MonPanneau panel = new MonPanneau();
    public MaFenetre(){
      this.setTitle("Premier dessin 2D")
      this.setDefaultCloseOperation(JFra
      this.add(panel);
      this.pack();//au lieu de this.setS
      this.setLocationRelativeTo(null);
      this.setVisible(true);
      }
  }
//ajout d'un Jpanel
class MonPanneau extends JPanel{
    public MonPanneau(){
        this.setPreferredSize(new Dimens
        }
    public void paint(Graphics g){//pair
      Graphics2D g2D = (Graphics2D) g;
      g2D.setStroke(new BasicStroke(5));
      g2D.setPaint(Color.pink);
      //g2D.drawRect(50, 50, 100, 200);
      g2D.fillRect(50, 50, 100, 200);
      }
  }
public class DessinerRectangle{
    public static void main(String[] arg
      MaFenetre F = new MaFenetre();
      }
  }
```

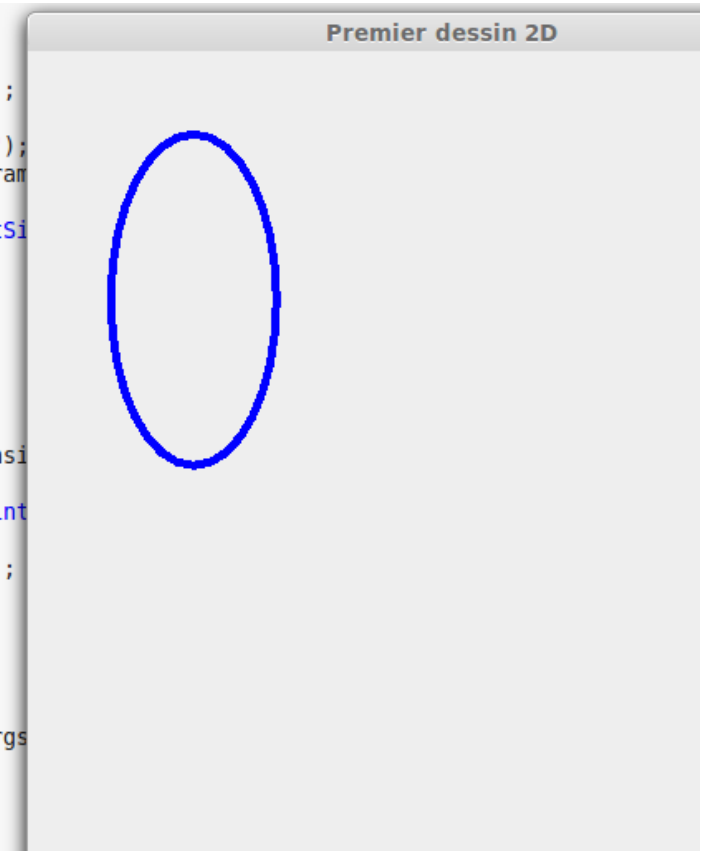


Dessiner un ovale (en particulier cercle)

```
class MonPanneau extends JPanel{
    public MonPanneau(){
        this.setPreferredSize(new Dimension(500,500));
        }
    public void paint(Graphics g){//paint est maintenant dans le
         panneau
      Graphics2D g2D = (Graphics2D) g;
      g2D.setStroke(new BasicStroke(5));
      g2D.setPaint(Color.blue);
      g2D.drawOval(50, 50, 100, 200);
      }
  }
```

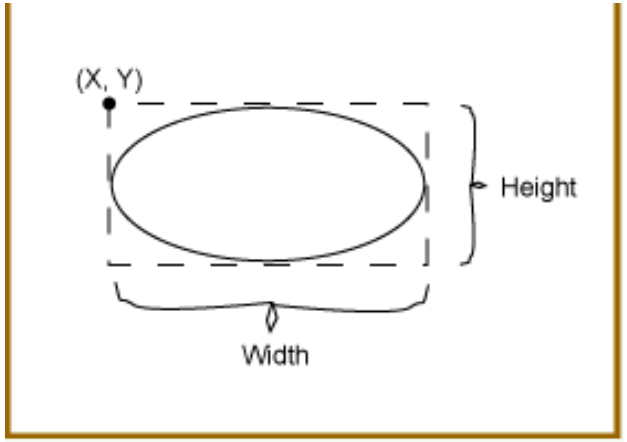
Exemple

```
import java.awt.*;import javax.swing.*;
class MaFenetre extends JFrame{
    MonPanneau panel = new MonPanneau();
    public MaFenetre(){
      this.setTitle("Premier dessin 2D");
      this.setDefaultCloseOperation(JFram
      this.add(panel);
      this.pack();//au lieu de this.setSi
      this.setLocationRelativeTo(null);
      this.setVisible(true);
      }
  }
//ajout d'un Jpanel
class MonPanneau extends JPanel{
    public MonPanneau(){
        this.setPreferredSize(new Dimensi
        }
    public void paint(Graphics g){//paint
      Graphics2D g2D = (Graphics2D) g;
      g2D.setStroke(new BasicStroke(5));
      g2D.setPaint(Color.blue);
      g2D.drawOval(50, 50, 100, 200);
      }
  }
public class DessinerOvale{
    public static void main(String[] args
      MaFenetre F = new MaFenetre();
      }
  }
```



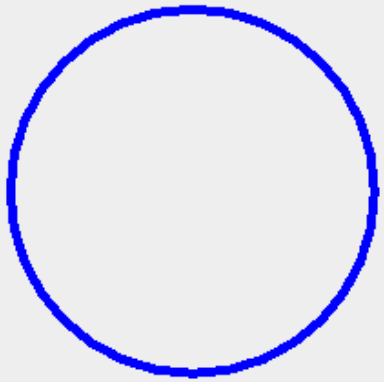
Remarque

- drawOval dessine un cercle ou une ellipse en spécifiant le rectangle dans lequel ils s'inscrivent ;
- Dans l'instruction `g2D.drawOval(50, 50, 200, 200);` si on donne la meme valeur pour la largeur et la longueur, alors on obtient un cercle de rayon cette valeur.



Dessiner un cercle

```
import java.awt.*;import javax.swing.*;
class MaFenetre extends JFrame{
    MonPanneau panel = new MonPanneau();
    public MaFenetre(){
      this.setTitle("Premier dessin 2D");
      this.setDefaultCloseOperation(JFrame.E
      this.add(panel);
      this.pack();//au lieu de this.setSize(
      this.setLocationRelativeTo(null);
      this.setVisible(true);
      }
  }
//ajout d'un Jpanel
class MonPanneau extends JPanel{
    public MonPanneau(){
        this.setPreferredSize(new Dimension(
        }
    public void paint(Graphics g){//paint es
      Graphics2D g2D = (Graphics2D) g;
      g2D.setStroke(new BasicStroke(5));
      g2D.setPaint(Color.blue);
      //g2D.drawOval(50, 50, 100, 200);
      g2D.drawOval(50, 50, 200, 200);
      }
  }
```

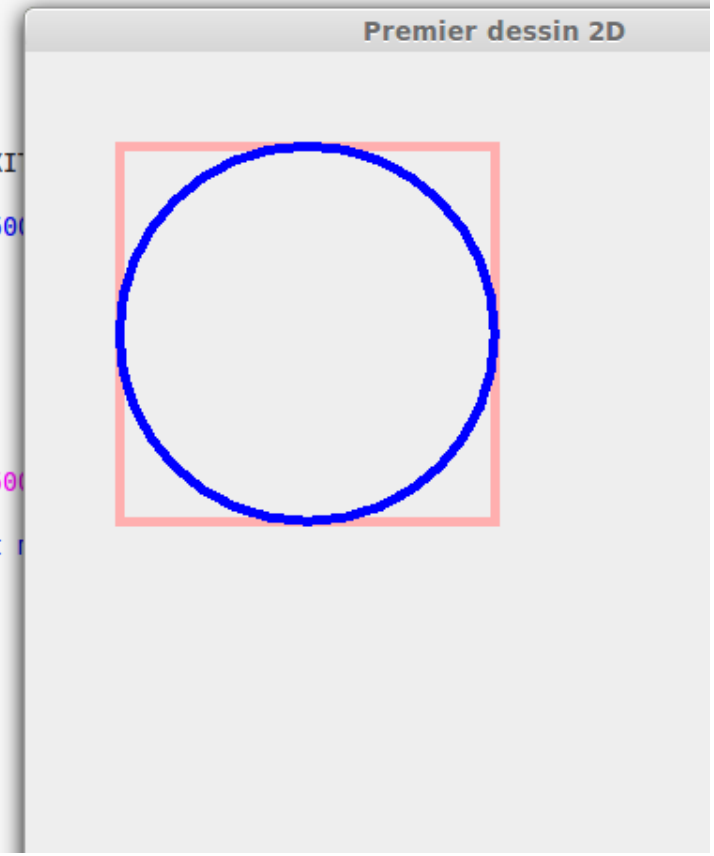


Dessiner un cercle entouré par un rectangle

```
class MonPanneau extends JPanel{
    public MonPanneau(){
        this.setPreferredSize(new Dimension(500,500));
        }
    public void paint(Graphics g){//paint est maintenant dans le
         panneau
      Graphics2D g2D = (Graphics2D) g;
      g2D.setStroke(new BasicStroke(5));
      g2D.setPaint(Color.pink);
      g2D.drawRect(50, 50, 100, 200);
      g2D.setPaint(Color.blue);
      g2D.drawOval(50, 50, 200, 200);
      }
  }
```

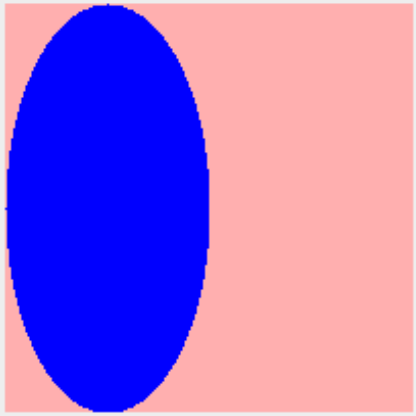
Dessiner un cercle entouré par un rectangle

```
import java.awt.*;import javax.swing.*;
class MaFenetre extends JFrame{
    MonPanneau panel = new MonPanneau();
    public MaFenetre(){
      this.setTitle("Premier dessin 2D");
      this.setDefaultCloseOperation(JFrame.EXI
      this.add(panel);
      this.pack();//au lieu de this.setSize(50
      this.setLocationRelativeTo(null);
      this.setVisible(true);
      }
  }
//ajout d'un Jpanel
class MonPanneau extends JPanel{
    public MonPanneau(){
        this.setPreferredSize(new Dimension(50
        }
    public void paint(Graphics g){//paint est
      Graphics2D g2D = (Graphics2D) g;
      g2D.setStroke(new BasicStroke(5));
      g2D.setPaint(Color.pink);
      g2D.drawRect(50, 50, 200, 200);
      g2D.setPaint(Color.blue);
      //g2D.drawOval(50, 50, 100, 200);
      g2D.drawOval(50, 50, 200, 200);
      }
  }
```



Dessiner un cercle entouré par un rectangle

```
    this.pack();//au lieu de this.setSize
    this.setLocationRelativeTo(null);
    this.setVisible(true);
    }
 }
/ajout d'un Jpanel
lass MonPanneau extends JPanel{
   public MonPanneau(){
       this.setPreferredSize(new Dimension
       }
   public void paint(Graphics g){//paint es
     Graphics2D g2D = (Graphics2D) g;
     g2D.setStroke(new BasicStroke(5));
     g2D.setPaint(Color.pink);
     //g2D.drawRect(50, 50, 200, 200);
     g2D.fillRect(50, 50, 200, 200);
     g2D.setPaint(Color.blue);
     //g2D.drawOval(50, 50, 100, 200);
     //g2D.drawOval(50, 50, 200, 200);
     g2D.fillOval(50, 50, 100, 200);
     }
 }
```



Dessiner un arc

- Un arc est dessiné comme une portion d'ovale. Les angles d'arc sont mesurés en degrés. Les arcs se déplacent le long d'une courbe à partir d'un angle de départ du nombre de degrés spécifié par leur angle d'arc.
- L'angle de départ indique en degrés où l'arc commence. L'angle de l'arc spécifie le nombre total de degrés parcourus par l'arc.
- La méthode `public void drawArc(int x, int y, int width, int height, int startAngle, int arcAngle)` Dessine un arc par rapport aux coordonnées *x* et *y* en haut à gauche du rectangle englobant avec la largeur et la hauteur spécifiées. Le segment d'arc est dessiné à partir de *startAngle* et balaie *arcAngle* degrés.

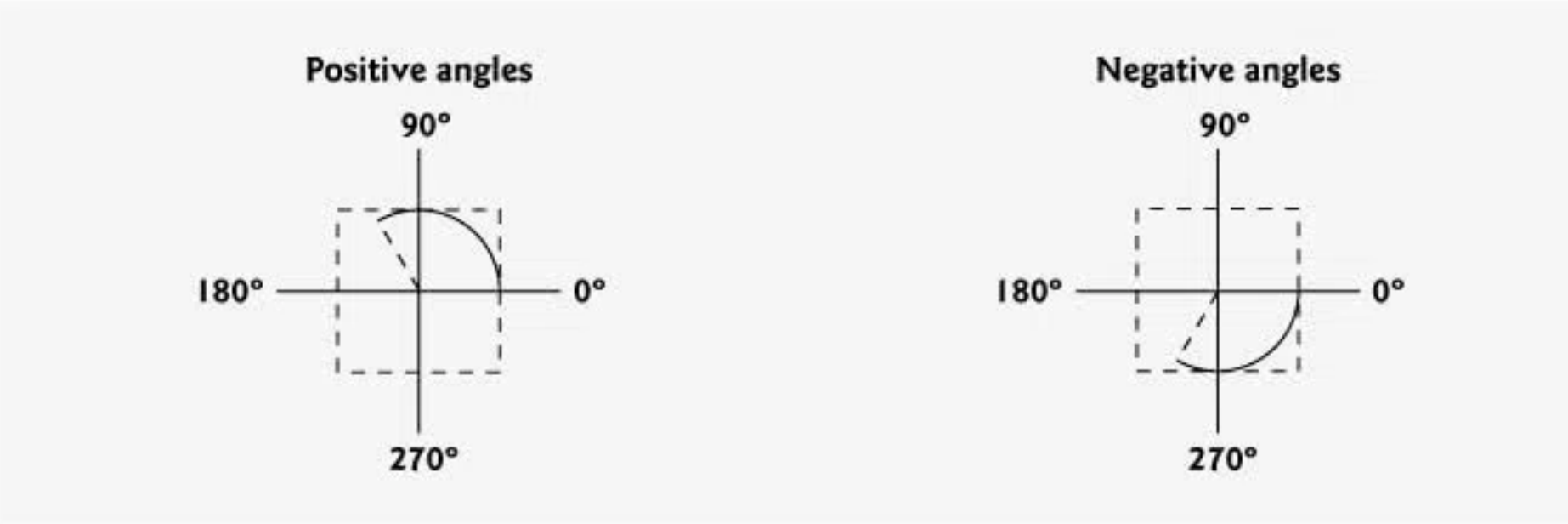
Dessiner un arc

- Un arc est dessiné comme une portion d'ovale. Les angles d'arc sont mesurés en degrés. Les arcs se déplacent le long d'une courbe à partir d'un angle de départ du nombre de degrés spécifié par leur angle d'arc.
- L'angle de départ indique en degrés où l'arc commence. L'angle de l'arc spécifie le nombre total de degrés parcourus par l'arc.
- La méthode `public void drawArc(int x, int y, int width, int height, int startAngle, int arcAngle)` Dessine un arc par rapport aux coordonnées *x* et *y* en haut à gauche du rectangle englobant avec la largeur et la hauteur spécifiées. Le segment d'arc est dessiné à partir de *startAngle* et balaie *arcAngle* degrés.

Dessiner un arc

- Un arc est dessiné comme une portion d'ovale. Les angles d'arc sont mesurés en degrés. Les arcs se déplacent le long d'une courbe à partir d'un angle de départ du nombre de degrés spécifié par leur angle d'arc.
- L'angle de départ indique en degrés où l'arc commence. L'angle de l'arc spécifie le nombre total de degrés parcourus par l'arc.
- La méthode `public void drawArc(int x, int y, int width, int height, int startAngle, int arcAngle)` Dessine un arc par rapport aux coordonnées *x* et *y* en haut à gauche du rectangle englobant avec la largeur et la hauteur spécifiées. Le segment d'arc est dessiné à partir de *startAngle* et balaie *arcAngle* degrés.

Exemple

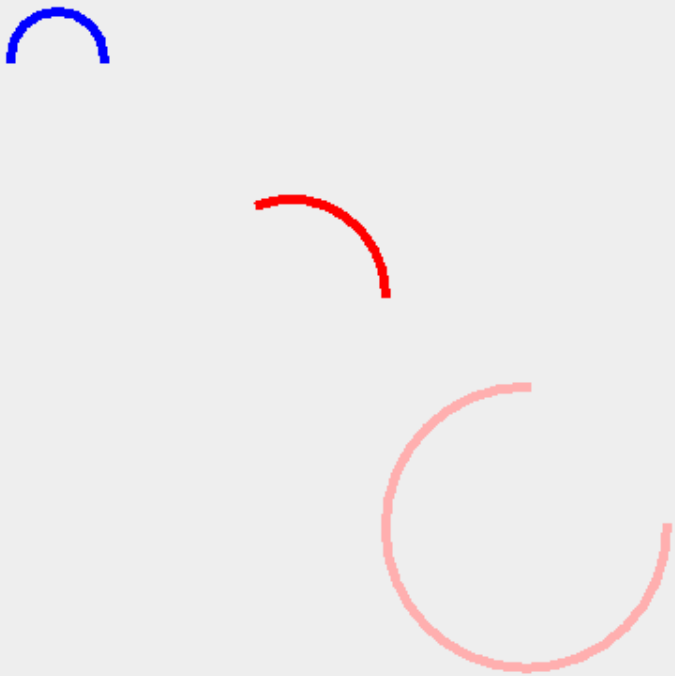


Dessiner un arc

```
class MonPanneau extends JPanel{
    public MonPanneau(){
        this.setPreferredSize(new Dimension(500,500));
        }
    public void paint(Graphics g){//paint est maintenant dans le
         panneau
      Graphics2D g2D = (Graphics2D) g;
      g2D.setStroke(new BasicStroke(5));
      g2D.setPaint(Color.blue);
      g2D.drawArc(50, 50, 50, 50,0,180);
      g2D.setPaint(Color.red);
      g2D.drawArc(150, 150, 100, 100,0,110);
      g2D.setPaint(Color.pink);
      g2D.drawArc(250, 250, 150, 150,0,-270);
      }
  }
```

Exemple

```
MonPanneau panel = new MonPanneau();
  public MaFenetre(){
    this.setTitle("Premier dessin 2D");
    this.setDefaultCloseOperation(JFrame.EXIT
    this.add(panel);
    this.pack();//au lieu de this.setSize(506
    this.setLocationRelativeTo(null);
    this.setVisible(true);
    }
 }
/ajout d'un Jpanel
lass MonPanneau extends JPanel{
  public MonPanneau(){
      this.setPreferredSize(new Dimension(506
      }
  public void paint(Graphics g){//paint est m
    Graphics2D g2D = (Graphics2D) g;
    g2D.setStroke(new BasicStroke(5));
    g2D.setPaint(Color.blue);
    g2D.drawArc(50, 50, 50, 50,0,180);
    g2D.setPaint(Color.red);
    g2D.drawArc(150, 150, 100, 100,0,110);
    g2D.setPaint(Color.pink);
    g2D.drawArc(250, 250, 150, 150,0,-270);
    }
 }
ublic class DessinerArc{
```

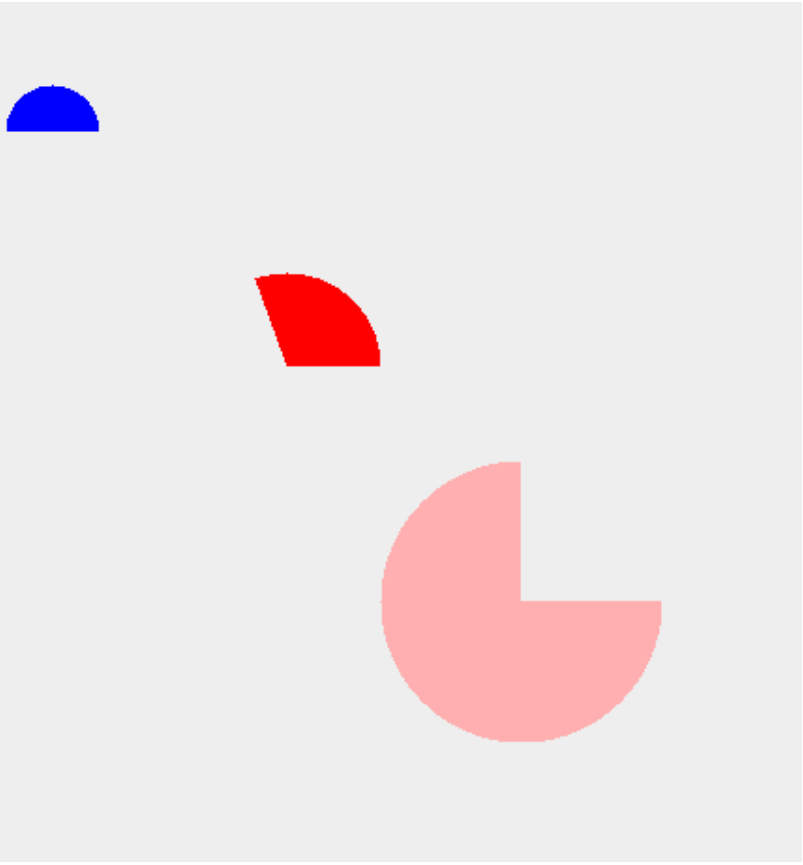


Dessiner un arc

```
class MonPanneau extends JPanel{
    public MonPanneau(){
        this.setPreferredSize(new Dimension(500,500));
        }
    public void paint(Graphics g){//paint est maintenant dans le
         panneau
      Graphics2D g2D = (Graphics2D) g;
      g2D.setStroke(new BasicStroke(5));
      g2D.setPaint(Color.blue);
      g2D.fillArc(50, 50, 50, 50,0,180);
      g2D.setPaint(Color.red);
      g2D.fillArc(150, 150, 100, 100,0,110);
      g2D.setPaint(Color.pink);
      g2D.fillArc(250, 250, 150, 150,0,-270);
      }
  }
```

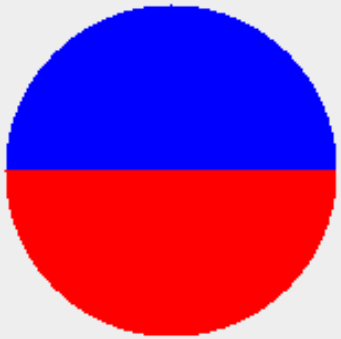
Exemple

```
import java.awt.*;import javax.swing.*;
class MaFenetre extends JFrame{
    MonPanneau panel = new MonPanneau();
    public MaFenetre(){
      this.setTitle("Premier dessin 2D");
      this.setDefaultCloseOperation(JFrame.EXIT_ON_
      this.add(panel);
      this.pack();//au lieu de this.setSize(500,50
      this.setLocationRelativeTo(null);
      this.setVisible(true);
      }
  }
//ajout d'un Jpanel
class MonPanneau extends JPanel{
    public MonPanneau(){
        this.setPreferredSize(new Dimension(500,50
        }
    public void paint(Graphics g){//paint est main
      Graphics2D g2D = (Graphics2D) g;
      g2D.setStroke(new BasicStroke(5));
      g2D.setPaint(Color.blue);
      g2D.fillArc(50, 50, 50, 50,0,180);
      g2D.setPaint(Color.red);
      g2D.fillArc(150, 150, 100, 100,0,110);
      g2D.setPaint(Color.pink);
      g2D.fillArc(250, 250, 150, 150,0,-270);
      }
```



Exercice

Réalisez le graphe suivant :



Solution

```
class MonPanneau extends JPanel{
    public MonPanneau(){
        this.setPreferredSize(new Dimension(500,500));
        }
    public void paint(Graphics g){
      Graphics2D g2D = (Graphics2D) g;
      g2D.setStroke(new BasicStroke(5));
      g2D.setPaint(Color.blue);
      g2D.fillArc(150, 150, 150, 150,0,180);
      g2D.setPaint(Color.red);
      g2D.fillArc(150, 150, 150, 150,0,-180);
      //ou bien g2D.fillArc(150, 150, 150, 150,180,180);
      }
  }
```